

CROSS-LINGUAL FAITHFULNESS EVALUATION OF RAG CHATBOTS

[AUTHOR NAME PLACEHOLDER]

[INSTITUTION PLACEHOLDER]

[GUIDE NAME PLACEHOLDER]

Abstract—This project evaluates the faithfulness of Retrieval-Augmented Generation (RAG) chatbots across multiple languages. Specifically, it assesses performance on English queries, Tamil-English code-mixed queries, and Hindi-English code-mixed queries. The study investigates whether query normalization and mitigation techniques can improve retrieval accuracy and answer faithfulness for code-mixed queries. This system functions as an evaluation framework using a controlled experimental design, and does not involve training or fine-tuning a predictive machine-learning model.

Index Terms—Retrieval-Augmented Generation, Faithfulness Evaluation, Code-Mixed Queries, Query Normalization, Large Language Models.

I. INTRODUCTION

Retrieval-Augmented Generation (RAG) architectures have emerged as a dominant paradigm for grounding Large Language Model (LLM) responses in external knowledge, thereby reducing hallucinations. However, the evaluation of these systems in multilingual and code-mixed environments remains a significant challenge. Code-mixing, the fluid alternation between two or more languages within a single utterance, is highly prevalent in diverse linguistic regions but severely challenges conventional retrieval systems built primarily for high-resource monolingual corpora.

This project presents a cross-lingual faithfulness evaluation framework for RAG chatbots. It specifically targets English, Tamil-English, and Hindi-English queries to determine the impact of query normalization on retrieval and answer generation fidelity. The core objective is to ascertain whether mitigating code-mixed queries into a standard language format prior to vector search enhances the overall faithfulness of the generated output.

Unlike predictive modeling pipelines, this system operates purely as an evaluation framework. It integrates retrieval, LLM-based generation, and an LLM-as-a-judge component to systematically score the adherence of answers to the retrieved contexts.

II. PROBLEM STATEMENT / EXISTING SYSTEM

Existing RAG evaluation pipelines predominantly focus on monolingual English benchmarks. When confronted with code-mixed queries—such as Tamil-English or Hindi-English—these systems frequently exhibit degraded retrieval recall because the semantic

representations of code-mixed tokens do not align well with the embedded knowledge base.

Consequently, if the retrieval stage fails to extract the correct context, the generation stage is prone to hallucination, regardless of the underlying LLM's capabilities. The problem necessitates an automated framework capable of evaluating whether normalizing these code-mixed queries before retrieval can restore faithfulness to acceptable levels.

III. PROPOSED METHODOLOGY

A. Overall System Architecture

The proposed methodology evaluates the end-to-end RAG pipeline using a controlled experimental design. The system ingests source documents, chunks the text, and computes dense embeddings to build a FAISS vector store. User queries undergo code-mix detection; if code-mixing is identified and mitigation is enabled, the query is normalized. The resulting query is used to retrieve the top-K relevant chunks, which are then passed to the LLM to generate an answer. Finally, an LLM-as-a-judge evaluates the generated answer against the retrieved context to yield a faithfulness score and a hallucination flag, which are stored in a SQLite database for subsequent statistical analysis.

B. Major Components

The primary modules include Document Processing, Multilingual Embedding, Vector Retrieval, Code-Mix Detection, Query Normalization, Answer Generation, and Faithfulness Evaluation. The evaluation module acts as the critical measuring instrument in this study.

C. Data Flow

The input documents flow through text chunking and embedding before being indexed. During testing, an input query is processed sequentially: Code-Mix Detection → Optional Query Normalization → Embedding → Top-K Retrieval → LLM Answer Generation → LLM-as-a-Judge Faithfulness Evaluation → Storage.

D. Technologies Used

The implementation leverages Python as the core programming language. The `intfloat/multilingual-e5-base` model via `SentenceTransformers` provides embedding capabilities. FAISS is utilized for fast vector retrieval. The Groq API hosts the `openai/gpt-oss-20b` model, which powers both the answer generation and the faithfulness

judge. Data persistence is handled by SQLite. The framework utilizes Pydantic for validation, SciPy for statistical analysis, and Pytest for automated testing.

E. Experimental Design

The experiment utilizes a paired design evaluating six discrete conditions across the same underlying information needs: (C1) English — Mitigation Disabled, (C2) Tamil-English — Mitigation Disabled, (C3) Hindi-English — Mitigation Disabled, (C4) English — Mitigation Enabled, (C5) Tamil-English — Mitigation Enabled, and (C6) Hindi-English — Mitigation Enabled.

IV. DATASET AND PREPROCESSING

A. Dataset / Data Source

The dataset comprises project-provided synthetic sample documents designed to simulate a real-world corpus. The current pilot corpus consists of 4 source files, which are parsed into 12 documents and further divided into 23 text chunks.

B. Query Dataset

The query dataset contains paired variants for specific underlying information needs. Each information need encompasses one English query, one Tamil-English query, and one Hindi-English query.

C. Number of Samples

The completed pilot dataset contains 3 underlying information needs, yielding 9 distinct query variants. Evaluated across the mitigation conditions, this resulted in 18 completed experimental runs. The planned full-scale study will expand to approximately 50 underlying information needs, 150 query variants, and 300 experimental runs.

D. Features / Attributes

Each experimental record encapsulates the question ID, the raw query text, language label, mitigation status, processed query string, retrieved contexts, retrieval scores, generated answer, faithfulness score, hallucination flag, and the judge's rationale.

E. Classes / Categories

Queries fall into three linguistic categories: English, Tamil-English, and Hindi-English. Operationally, they are evaluated under two categories: Baseline (Mitigation Disabled) and Mitigated (Mitigation Enabled).

F. Data Collection Method

Queries and document sources were synthetically formulated to tightly control the domain semantics and test specific multilingual retrieval challenges.

G. Data Preprocessing

Source documents are loaded and cleaned to remove whitespace and formatting inconsistencies, then chunked using a sliding window approach with preserved metadata.

H. Training, Testing and Validation Consideration

A conventional machine learning training, testing, and validation split is not applicable. The system does not train or fine-tune predictive models; it is an evaluation framework utilizing a controlled paired experimental design to assess system responses.

V. IMPLEMENTATION

A. Development Environment

The framework is developed in Python, utilizing Git and GitHub for version control.

B. Document Ingestion

Source documents are read, cleaned, and chunked while carefully preserving associated metadata.

C. Embedding Generation

The chunked texts are encoded into dense vectors using the `intfloat/multilingual-e5-base` embedding model.

D. FAISS Retrieval

The system constructs a FAISS index from the document embeddings, enabling efficient Top-K retrieval based on query cosine similarity.

E. Code-Mix Detection

A detection module identifies the linguistic composition of queries, distinguishing between monolingual English, Tamil-English, and Hindi-English inputs based on lexical and script heuristics.

F. Query Normalization / Mitigation

When mitigation is enabled, code-mixed queries are translated and normalized into English. The process strictly preserves named entities, numbers, dates, product

names, restrictions, and the intended constraints. English queries bypass this rewriting phase.

G. RAG Answer Generation

The LLM, powered by the Groq API, synthesizes an answer utilizing only the context provided by the retrieved Top-K chunks.

H. Faithfulness Evaluation

An LLM-as-a-judge assesses the generated response against the retrieved context, returning a binary hallucination flag and a continuous faithfulness score bounded between 0 and 1, along with a rationale.

I. SQLite Result Storage

All experimental runs, including context, queries, scores, and rationales, are logged into a SQLite database for auditing and statistical evaluation.

J. Statistical Analysis

The framework calculates descriptive statistics including the mean, median, standard deviation, minimum, maximum, and hallucination rate. It employs the paired Wilcoxon signed-rank test to calculate p-values for condition comparisons.

K. Automated Testing

To ensure robust software validation, the implementation includes a comprehensive Pytest suite with 54 out of 54 automated tests currently passing. This testing verifies implementation correctness, separate from the research experiment results.

L. GitHub Repository

The complete project implementation, scripts, tests, documentation, and configuration files are maintained in the project repository at [GITHUB URL PLACEHOLDER]. API keys and secrets are strictly excluded from version control.

VI. EXPERIMENTATION AND RESULTS

A. Experimental Setup

The pilot experiment executed 18 test runs evaluating the 3 underlying information needs across the 6 paired conditions. All LLM calls utilized the openai/gpt-oss-20b model hosted on Groq.

B. Working Demonstration

The working demonstration exposes the complete pipeline. A user query enters the system, undergoes code-mix detection, and is optionally normalized. The processed query is embedded and used for FAISS retrieval. The retrieved context feeds the LLM for answer generation, which is then scored by the faithfulness judge and logged to the SQLite database.

[FIGURE PLACEHOLDER – English Query]

[FIGURE PLACEHOLDER – Tamil-English Query]

[FIGURE PLACEHOLDER – Hindi-English Query]

[FIGURE PLACEHOLDER – Retrieved Context]

[FIGURE PLACEHOLDER – Generated Answer]

[FIGURE PLACEHOLDER – Faithfulness Evaluation]

[FIGURE PLACEHOLDER – SQLite Results]

C. Evaluation Metrics

Primary metrics include Mean Faithfulness, Median Faithfulness, Standard Deviation, Hallucination Rate, Minimum Faithfulness, and Maximum Faithfulness. Recall@K is not reported in the current pilot. A paired Wilcoxon signed-rank test is utilized to measure the statistical difference between baseline and mitigated conditions.

D. Experimental Results

The pilot phase completed 18 experimental runs with 0 failures. The Mean faithfulness across runs was 1.000, with a Median of 1.000, and a Standard Deviation of 0.000. The Hallucination rate was 0/18 (0%). The English baseline mean was 1.000. Both the Tamil-English mitigated mean and the Hindi-English mitigated mean were 1.000.

E. Statistical Analysis

Due to the perfect faithfulness scores (all paired differences were exactly zero), the paired Wilcoxon signed-rank tests were not testable in the current pilot phase.

F. Success Criterion

The predefined success criterion requires that the difference between the English baseline mean and the

mitigated code-mixed mean is less than or equal to 0.10. Based on the pilot results, this criterion was MET for both Tamil-English and Hindi-English queries.

G. Limitations

These outcomes represent pilot results restricted to 3 information needs and 18 runs, and should not be generalized as final full-scale results. The lack of variance in the pilot scores precluded full statistical hypothesis testing.

VII. CONCLUSION

This project successfully implemented an end-to-end faithfulness evaluation framework for cross-lingual RAG chatbots. The pilot results indicate perfect baseline adherence and successful mitigation of code-mixed queries for the limited sample set. The automated pipeline establishes a robust foundation for the planned full-scale study, which will further investigate the bounds of query normalization across larger code-mixed query volumes.

REFERENCES

- [1] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*.
- [2] Reimers, N., and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*.
- [3] Johnson, J., Douze, M., and Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*.
- [4] Zheng, L., et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*.
- [5] Wang, L., et al. (2023). Faithfulness Evaluation of Retrieval-Augmented Generation. *arXiv preprint*.